

# PMAD • Predictive Memory Allocator

## Техническа Документация v1.0

Проектирано и разработено от

**Димитър Георгиев Анастасов**

|                                                                                               |    |
|-----------------------------------------------------------------------------------------------|----|
| 1. Тема .....                                                                                 | 2  |
| 2. Автор .....                                                                                | 2  |
| 3. Ръководител .....                                                                          | 2  |
| 4. Резюме .....                                                                               | 3  |
| 4.1. Цели (предназначение, анализ на потребностите и съществуващите решения) ....             | 3  |
| 4.1.1. Предназначение .....                                                                   | 3  |
| 4.1.2. Анализ на потребностите .....                                                          | 4  |
| 4.1.3. Съществуващи решения .....                                                             | 4  |
| 4.2. Основни етапи в реализирането на проекта .....                                           | 6  |
| 4.2.1. Обучение и изследване: .....                                                           | 7  |
| 4.2.2. Проектиране .....                                                                      | 7  |
| 4.2.3. Реализация .....                                                                       | 7  |
| 4.3 Ниво на сложност на проекта – основни проблеми при реализацията на поставените цели ..... | 7  |
| 4.3.1. Общ преглед на сложността на проекта .....                                             | 8  |
| 4.3.2. Гарантиране на строго $O(1)$ време за изпълнение .....                                 | 8  |
| 4.3.3. Подравняване на паметта .....                                                          | 8  |
| 4.3.4. Минимизиране на режимните разходи .....                                                | 9  |
| 4.3.5. Предварителна пресмятаемост на ресурсите .....                                         | 9  |
| 4.3.6. Изолация от операционната система .....                                                | 10 |
| 4.3.7. Защита от грешки и препълване на буфера .....                                          | 10 |
| 4.4. Логическо и функционално описание на решението .....                                     | 10 |
| 4.4.1. Архитектура на алакатора .....                                                         | 10 |
| 4.4.2. Структури от данни .....                                                               | 12 |
| 4.4.2.1. MemoryPool .....                                                                     | 12 |
| Описание .....                                                                                | 12 |
| Размер на структурата .....                                                                   | 12 |
| 4.4.2.2 BlockHeader .....                                                                     | 12 |
| Описание .....                                                                                | 12 |
| Размер на структурата .....                                                                   | 12 |
| 4.4.2.2 SizeClass .....                                                                       | 13 |
| Описание .....                                                                                | 13 |
| Размер на структурата .....                                                                   | 13 |

|                                                                                                                            |    |
|----------------------------------------------------------------------------------------------------------------------------|----|
| 4.4.2.2 PMAD .....                                                                                                         | 13 |
| Описание .....                                                                                                             | 13 |
| Размер на структурата .....                                                                                                | 13 |
| 4.4.3 Макроси и константи .....                                                                                            | 13 |
| 4.4.4 Инициализация на алокатора .....                                                                                     | 14 |
| 4.4.4.1 PMAD_init() .....                                                                                                  | 14 |
| 4.4.4.2 build_lookup_table() .....                                                                                         | 15 |
| 4.4.4.3 Пример на таблицата .....                                                                                          | 15 |
| 4.4.5. Взаимодействие с операционната система .....                                                                        | 16 |
| 4.4.5.1. get_memory_pool_from_os() .....                                                                                   | 16 |
| 4.4.5.2. free_memory_pool() .....                                                                                          | 17 |
| 4.4.6. Организация на вътрешната памет .....                                                                               | 17 |
| 4.4.6.1. attach_new_pool() .....                                                                                           | 17 |
| 4.4.6.2. split_pool_by_percentage() .....                                                                                  | 18 |
| Гъвкавост на конфигурацията .....                                                                                          | 18 |
| Обща логическа организация .....                                                                                           | 18 |
| createBlock() .....                                                                                                        | 19 |
| 4.4.7. Алокация и освобождаване .....                                                                                      | 20 |
| 4.4.7.1. Алокация .....                                                                                                    | 20 |
| 4.4.7.2. Функцията roundUp() .....                                                                                         | 20 |
| 4.4.7.3. Пример на функциониране .....                                                                                     | 20 |
| 4.4.7.4. Освобождаване на памет .....                                                                                      | 21 |
| 4.4.8. Обвивка и използване на алокатора .....                                                                             | 21 |
| 4.4.8.1. Обвивка на алокатора (incPMAD) .....                                                                              | 22 |
| 4.5. Реализация – обосновка за използвани технологични средства, алгоритми,<br>литература, програмни приложения и др. .... | 22 |
| 4.5.1 Използвани технологични средства (Език и среда) Език за програмиране C<br>(C99/C11) .....                            | 22 |
| POSIX API (<sys/mman.h>, <time.h>) .....                                                                                   | 23 |
| Компилятор и build система .....                                                                                           | 23 |
| Развойна среда и дебъгване .....                                                                                           | 23 |
| 4.5.2. Използвани алгоритми и структури от данни .....                                                                     | 23 |
| Slab Allocation (Алокация базирана на плочи) .....                                                                         | 24 |

|                                                                                |    |
|--------------------------------------------------------------------------------|----|
| LIFO Free List (Едносвързан списък като стек) .....                            | 24 |
| O(1) Lookup Table .....                                                        | 24 |
| Побитова аритметика за подравняване .....                                      | 24 |
| 4.5.3 Програмни приложения и инструменти за анализ Бенчмарк система .....      | 25 |
| Визуализация (Web технологии) .....                                            | 25 |
| 4.5.4 Използвана литература и източници .....                                  | 26 |
| 4.6. Описание на приложението – инсталация, стартиране, използване и поддръжка | 26 |
| Инсталация и стартиране .....                                                  | 26 |
| Интеграция и използване .....                                                  | 27 |
| 1. Инициализация .....                                                         | 28 |
| 2. Заделяне и освобождаване на памет .....                                     | 28 |
| 3. Унищожаване на алокатора .....                                              | 29 |
| Пълен пример (алокация на масив от 6 елемента, принтиране и освобождаване) ..  | 29 |
| Поддръжка .....                                                                | 30 |
| 4.7. Заключение .....                                                          | 31 |
| 4.7.1. Основен резултат .....                                                  | 31 |
| 4.7.2. Експериментални резултати .....                                         | 31 |
| 4.7.3. Визуализация на резултатите .....                                       | 32 |
| 4.7.4. Текущи приложения .....                                                 | 34 |
| 4.7.5. Възможности за бъдещо развитие .....                                    | 34 |
| 4.7.5.1. Поддръжка на многонишковост .....                                     | 34 |
| 4.7.5.2. Динамично разширяване на пула .....                                   | 34 |
| 4.7.5.3. Memory Safety механизми .....                                         | 35 |
| 4.7.5.4. Разширено подравняване .....                                          | 35 |
| 4.7.6 Финална оценка .....                                                     | 35 |

# 1.Тема

PMAD е детерминистичен C алокатор на памет с конфигурируеми размерни класове и процентно разпределение, осигуряващ **O(1) алокация и освобождаване**, без системни извиквания по време на работа. Подходящ е за real-time, game engines, мрежови системи, вградени системи, и всичко за което потребителя реши да го конфигурира, като предлага напълно предсказуемо поведение и предварително изчислими ресурси и производителност.

## 2.Автор

Три имена: Димитър Георгиев Анастасов

ЕГН: 0848116280

Адрес: Улица средна гора 57 гр. София

Телефон: 0887383000

Емейл: [dimitar.anastassow@gmail.com](mailto:dimitar.anastassow@gmail.com)

Училище: ТУЕС

Клас: 11<sup>В</sup>

## 3.Ръководител

Три имена: Кристиан Руменов Стоименов

Телефон: 0889991139

Имейл: [kstoimenov@elsys-bg.org](mailto:kstoimenov@elsys-bg.org)

Длъжност: Системен програмист

## 4. Резюме

### 4.1. Цели (предназначение, анализ на потребностите и съществуващите решения)

#### 4.1.1. Предназначение

PMAD е създаден с целта да предостави лек, напълно детерминистичен и конфигурируем алокатор на памет, подходящ за среди, в които предвидимостта и скоростта на управлението на паметта са от критично значение. Алокаторът е предназначен за приложения, при които стандартните решения (malloc/free и тн.) не могат да гарантират постоянно време за изпълнение — например системи за работа в реално време (real-time systems), вградени системи (embedded/RTOS), game engines и системи за високочестотна търговия (HFT). Основните цели на проекта са:

- **Строга  $O(1)$  сложност** за заделяне и освобождаване на памет — не амортизирана, а гарантирана за всяка отделна операция
- **Нулеви системни извиквания** по време на работа — цялата памет се заделя предварително при инициализация чрез единствено извикване на `mmap()`, а по време на работа алокаторът не взаимодейства с ядрото на операционната система
- **Пълна конфигурируемост** — потребителят сам определя размерите на класовете блокове и процентното разпределение на паметта между тях
- **100% предвидимост преди изпълнение** — броят на блоковете, използваната памет и overhead-ът могат да бъдат математически изчислени преди да бъде написан дори един ред приложен код.

### 4.1.2. Анализ на потребностите

Всъвременните софтуерни системи управлението на паметта е ключов фактор за производителността и надеждността. Стандартният C алокатор (malloc/free от glibc, ptmalloc) има няколко съществени ограничения:

- **Непредсказуема латентност** — ptmalloc използва системни извиквания (brk, mmap), които могат да отнемат от микросекунди до милисекунди в зависимост от натоварването на системата. Средната латентност за малки обекти е  $\sim 300$  ns, но в най-лошия случай може да достигне десетки микросекунди;
- **Липса на строг детерминизъм** — операциите са амортизирано  $O(1)$ , но отделни операции могат да бъдат значително по-бавни поради сливане и разделяне на блокове или системни извиквания, което прави поведението непредсказуемо;
- **Ограничен контрол върху ресурсите** — потребителят не може предварително да определи броя налични блокове, организацията на паметта или режимните разходи, тъй като те зависят от динамичното поведение на програмата;
- **Фрагментация и jitter** — продължителната работа с различни размери алокации води до вътрешна и външна фрагментация, както и до големи разлики между най-бързата и най-бавната операция, което е проблем за реалновремеви системи.

Тези проблеми са особено критични във вградени системи, игрови двигатели, високочестотна търговия и телекомуникационни сървъри, където се изисква предсказуемо време за изпълнение.

### 4.4.3. Съществуващи решения

Съществуващите алокатори частично решават тези проблеми, но не предоставят едновременно конфигурируемост, детерминизъм и предварителна изчислимост на ресурсите.

ptmalloc — стандартният Linux алокатор. Използва арени и динамични bins с амортизирано  $O(1)$  време, но страда от непредсказуема латентност и липса на контрол върху вътрешната структура на паметта.

jemalloc — предлага по-добра производителност ( $\sim 30\text{--}60$  ns за малки обекти) и по-малка фрагментация чрез 71 фиксирани размерни класа и thread-local кешове. Въпреки това размерните класове са неизменяеми и липсва предварителна пресмятаемост на ресурсите.

tcmalloc — използва per-thread кешове и около 80 фиксирани размерни групи, осигурявайки висока пропускателна способност ( $\sim 600\text{--}1000$  М алокации/сек). Подобно на jemalloc, не позволява пълен контрол върху разпределението на паметта.

TLSF — реалновременен алокатор с  $O(1)$  операции чрез двустепенно bitmap индексирание. Въпреки това не предлага добра предварителна планируемост на ресурсите и има по-висока сложност на имплементация.

PMAD се отличава чрез по-голяма конфигурируемост, точна и 100 процентова предсказуемост на ресурсите и възможност за предварително изчисляване на използваната памет, което го прави по-подходящ за системи със строги изисквания за детерминизъм.



## PMAD vs jemalloc vs tcmalloc vs ptmalloc

PMAD values are measured. Competitor values are from published benchmarks (marked ~). The configurability column is binary — either an allocator supports it or it doesn't.

| Metric                         | PMAD (measured)              | jemalloc (~approx)   | tcmalloc (~approx)  | ptmalloc (~approx) |
|--------------------------------|------------------------------|----------------------|---------------------|--------------------|
| Alloc complexity               | 0(1) strict ✓                | 0(1) amortised       | 0(1) amortised      | 0(1) amortised     |
| Alloc path ops                 | 4 ops (proven)               | ~10-20 ops           | ~10-15 ops          | ~30-50 ops         |
| Latency — small obj.           | 33.3 ns avg (measured)       | ~30-60 ns            | ~50 ns              | ~300 ns            |
| Bulk throughput                | 468 M alloc/s (measured)     | ~500-900 M/s         | ~600-1000 M/s       | ~80-150 M/s        |
| Runtime syscalls               | 0 (post-init)                | periodic mmap        | periodic mmap/sbrk  | frequent brk/mmap  |
| Total syscalls (lifecycle)     | 2 (1 mmap + 1 munmap)        | N on-demand          | N on-demand         | N+ per arena       |
| User-configurable size classes | ✓ Yes — full control         | ✗ Fixed (71 classes) | ✗ Fixed (~80 bands) | ✗ Dynamic bins     |
| User-configurable % per class  | ✓ Yes — exact split          | ✗ No                 | ✗ No                | ✗ No               |
| Pre-runtime block count known  | ✓ Mathematically exact       | ✗ Not knowable       | ✗ Not knowable      | ✗ Not knowable     |
| Determinism / zero jitter      | ✓ Fully deterministic        | Mostly predictable   | Mostly predictable  | Non-deterministic  |
| Per-block overhead             | 16 B (BlockHeader, measured) | ~8-16 B avg          | ~8-24 B avg         | ~16-32 B avg       |
| Header overhead %              | 18.35% (measured)            | ~5-15%               | ~15-25%             | ~10-20%            |
| Max alloc size                 | 4096 B (configurable)        | Unlimited            | Unlimited           | Unlimited          |
| Thread safety                  | Single-threaded              | Lock-free TLS        | Lock-free TLS       | Arena-per-thread   |
| RTOS / real-time suitability   | ✓ Yes                        | ✗ No                 | ✗ No                | ✗ No               |

PMAD latency marked (measured). Competitor figures sourced from: ithare.com, AppFolio Engineering, tcmalloc.dev, jemalloc.net. Competitor numbers vary by hardware/workload.

Фигура 1: Таблица със сравнения с най-добрите подобни алокатори

## 4.2. Основни етапи в реализирането на проекта

### 4.2.1. Обучение и изследване:

Обучението включваше подробен преглед на управлението на паметта във всяка операционна система, включително работа с виртуална памет, RAM и дисково пространство. Използвана беше специализирана техническа литература и документация от утвърдени източници като технически ръководства и документации от компании като Intel, както и други ресурси, свързани с архитектурата и управлението на системна памет. След усвояване на основните концепции беше проведено изследване на проблемите, които възникват при практическата реализация и използването на различни подходи за управление на паметта.

### 4.2.2. Проектиране

На този етап се извършваше постепенно подобряване и доразвиване на идеята, както и уточняване и оптимизиране на архитектурата на системата с цел по-добра ефективност и надеждност.

### 4.2.3. Реализация

Фокусът беше върху имплементирането на проекта, като паралелно с това се мислеше за бъдещи подобрения, както и за улесняване работата на крайния потребител при използването на системата.

## 4.3 Ниво на сложност на проекта – основни проблеми при реализацията на поставените цели

### 4.3.1. Общ преглед на сложността на проекта

Проектът PMAD се характеризира с висока степен на сложност, тъй като изисква прецизно управление на системни ресурси и стриктно спазване на алгоритмични гаранции за време и памет. Основните предизвикателства са свързани с постигане на детерминистично поведение, минимизиране на режимните разходи и осигуряване на надеждна защита на паметта.

### 4.3.2. Гарантиране на строго $O(1)$ време за изпълнение

Една от най-големите трудности при разработката на детерминистични алокатори е елиминирането на вариациите в производителността (jitter).

При стандартните алгоритми за търсене на свободен блок, като First-Fit и Best-Fit, сложността може да нарасне до  $O(n)$  при силна фрагментация. В PMAD този проблем е решен чрез комбинация от Lookup таблица и slab алокационен подход. Чрез предварително изчислена таблица за съответствие между размер и клас памет (`size_class_reference`, описана в [раздел 4.4.4](#)) се осигурява намиране на подходящия клас памет чрез една операция, независимо от натоварването на системата. Това свежда процеса на алокация до няколко атомарни операции.

### 4.3.3. Подравняване на паметта

На 64-битови архитектури неправилното подравняване на паметта може да доведе до влошена производителност или грешки при работа със SIMD инструкции.

Този проблем е решен чрез използване на 16-байтово подравняване чрез побитова операция: `(size + 15) & ~(15)`

Предизвикателството се състои в правилното изчисляване на отместването за структурата BlockHeader, описана в [раздел 4.4.2.2](#), така че потребителските данни да бъдат правилно подравнени непосредствено след заглавната информация.

#### 4.3.4. Минимизиране на режимните разходи

Всеки заделен блок памет съдържа допълнителна метайнформация. При малки блокове това може да доведе до значителен процент режимни разходи.

За решаването на този проблем е проектирана минималистична структура от 16 байта, която съдържа само най-необходимата информация — индекс на клас памет и указател към следващия свободен блок. Основното предизвикателство е постигането на баланс между достатъчно метаданни за бързо освобождаване на паметта и минимално използване на допълнителна памет.

#### 4.3.5. Предварителна пресмятаемост на ресурсите

Повечето стандартни алокатори не позволяват точно прогнозиране на използваните ресурси преди изпълнение.

В PMAD този проблем е решен чрез математическо моделиране на разделянето на паметния пул чрез функцията `split_pool_by_percentage`, описана в [раздел 4.4.6.2](#). Това позволява предварително изчисляване на броя налични блокове, което е критично за системи със строги изисквания за надеждност.

#### 4.3.6. Изолация от операционната система

Стандартното заделяне на памет често води до системни извиквания към ядрото, които могат да причинят закъснения и непредсказуемост.

PMAD използва предварително резервиране на памет (*eager allocation*), като след фазата на инициализация алокаторът работи изцяло в потребителското пространство, минимизирайки зависимостта от операционната система.

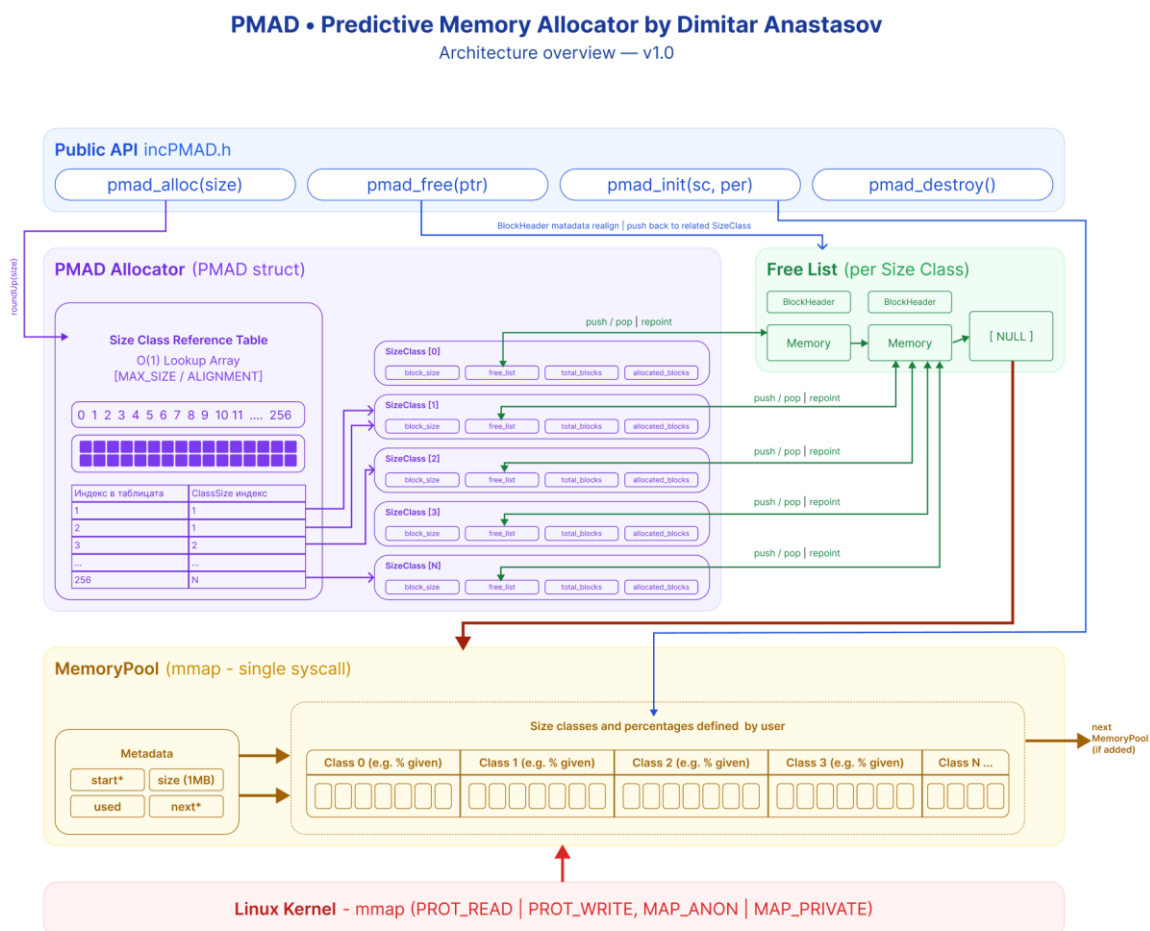
#### 4.3.7. Защита от грешки и препълване на буфера

При използване на фиксирани размерни класове съществува риск от заявки за памет извън допустимите граници.

За целта са добавени защитни проверки в функциите `build_lookup_table`, описана в [раздел 4.4.4.2](#) и `PMAD_alloc`, описана в [раздел 4.4.7.1](#), които връщат `NULL` при некоректни заявки и предотвратяват критични системни грешки.

## 4.4. Логическо и функционално описание на решението

### 4.4.1. Архитектура на алакатора



Фигура 2: PMAD архитектура

PMAD алокаторът е реализиран чрез йерархична архитектура, при която системата е разделена на логически независими компоненти. Комуникацията между компонентите е еднопосочна — от публичния API към вътрешното управление на паметта и към операционната система.

Архитектурата включва следните основни нива:

- Публичен интерфейс (API слой)
- Логика за управление на алокациите
- Управление на паметни пулове
- Системен слой за комуникация с операционната система

Тази архитектура позволява висока производителност, детерминистично поведение и лесна разширяемост на системата.

## 4.4.2. Структури от данни

### 4.4.2.1. MemoryPool

#### Описание

MemoryPool представлява голям блок памет, получен от операционната система, който служи като основен източник на памет за аллокатора.

#### Размер на структурата

В 64-битова архитектура структурата MemoryPool заема 32 байта, като всички полета са естествено подравнени за оптимална производителност.

#### 4.4.2.2 BlockHeader

##### Описание

BlockHeader представлява заглавката на всеки блок памет. Използва се за управление на свободните блокове и за свързването им в списъците на съответните SizeClass структури.

##### Размер на структурата

В 64-битова архитектура структурата BlockHeader заема 16 байта. Подравняването гарантира бърз достъп до следващия блок в списъка.

#### 4.4.2.2 SizeClass

##### Описание

SizeClass съдържа информация за блокове с еднакъв размер и управлява свободните блокове чрез свързан списък. Тази структура позволява бързо намиране на подходящ блок памет за алокация.

##### Размер на структурата

В 64-битова архитектура структурата SizeClass заема приблизително 48 байта, като полетата са подравнени за по-добра производителност.

#### 4.4.2.2 PMAD

##### Описание

PMAD представлява главната структура на алокатора и съдържа всички MemoryPool и SizeClass структури. Чрез нея се управляват всички операции по алокация и освобождаване на памет.

##### Размер на структурата

В 64-битова архитектура PMAD заема около 200–256 байта, в зависимост от броя на SizeClass елементите. Подравняването на структурата подобрява кеш ефективността.



### 4.4.3 Макроси и константи

**#define** **NUM\_CLASSES** **5**

Определя броя на SizeClass-овете в PMAD. Използва се от PMAD.size\_classes за управление на различните класове памет в алокатора.

**#define** **POOL\_SIZE** **(1024 \* 1024)**

Размер на всеки MemoryPool, който се извлича от операционната система. Използва се във функциите get\_memory\_pool\_from\_os(), описана в [раздел 4.4.5.1](#) и free\_memory\_pool(), описана в [раздел 4.4.5.2](#) за управление на системната памет.

**#define** **ALIGNMENT** **16**

Минималното изисквано подравняване на блоковете за алокация (за x86 Linux и macOS). Гарантира правилна работа с хардуерното подравняване и SIMD инструкции.

**#define** **MAX\_SIZE\_OF\_SIZE\_CLASS** **4096**

Максималният размер, който ще се поддържа за SizeClass. Използва се за изграждане на таблицата size\_class\_reference за бързо намиране на подходящ клас памет.

### 4.4.4 Инициализация на алокатора

#### 4.4.4.1 PMAD\_init()

**PMAD\_init()** отговаря за инициализацията на структурата PMAD и подготовката на всички SizeClass-ове за разпределение на памет.

Основните стъпки на функцията са следните:

- Задава размери на блоковете за всеки SizeClass. Размерите могат да се променят според нуждите на приложението.
- Инициализира списъка на свободните блокове (free\_list) за всеки SizeClass като празен.

- Нулира броя на общо създадените блокове (`total_blocks`) и броя на заетите блокове (`allocated_blocks`) за всеки `SizeClass`.
- Инициализира `pool_head` на `PMAD` като `NULL`, което означава, че все още няма създадена памет.
- Създава се таблица `size_class_reference` ([Фигура 3](#)) за бързо определяне на подходящия `SizeClass` при извикване на `PMAD_alloc()`. Таблицата се изгражда чрез `build_lookup_table()`.

#### 4.4.4.2 `build_lookup_table()`

**`build_lookup_table()`** има за цел да създаде таблица за бързо намиране на подходящия `SizeClass` по даден размер на заявката.

- Таблицата съдържа максимум

**`MAX_SIZE_OF_SIZE_CLASS / ALIGNMENT = 4096 / 16 = 256`** елемента.

- Всеки елемент съдържа индекса на най-малкия `SizeClass`, който е достатъчно голям, за да побере размера:

`size_class_reference[i] = индекс на най-малкия SizeClass  $\geq i \cdot \text{ALIGNMENT}$`

- Таблицата се изгражда линейно, като се преминава през всички подравнени размери от `ALIGNMENT` до `MAX_SIZE_OF_SIZE_CLASS`, а `class_index` се движи напред без да се връща назад.

#### 4.4.4.3 Пример на таблицата

Нека имаме следните `SizeClass`-ове:

`size_classes = [32, 64, 128, 256, 512]`

С подравняване  $\text{ALIGNMENT} = 16$ , таблицата `size_class_reference` ([Таблица 3](#)) ще изглежда по следния начин:

| Индекс $i$ | Подравнен размер $i \cdot \text{ALIGNMENT}$ | SizeClass индекс |
|------------|---------------------------------------------|------------------|
| 1          | 16                                          | 0                |
| 2          | 32                                          | 0                |
| 3          | 48                                          | 1                |
| 4          | 64                                          | 1                |
| 5          | 80                                          | 2                |
| 6          | 96                                          | 2                |
| 7          | 112                                         | 2                |
| 8          | 128                                         | 2                |
| 9          | 144                                         | 3                |
| 10         | 160                                         | 3                |
| ...        | ...                                         | ...              |
| 32         | 512                                         | 4                |

*Фигура 3: Визуално-логическа репрезентация на `size_class_reference`*

Така `PMAD_alloc()` може да намери подходящия `SizeClass` за всяка заявка в константно време  $O(1)$ .

#### 4.4.5. Взаимодействие с операционната система

##### 4.4.5.1. `get_memory_pool_from_os()`

Функцията `get_memory_pool_from_os()` използва системното извикване `mmap()` за алокация на нов `MemoryPool` с размер `POOL_SIZE`.

- Ако алокацията не успее, функцията извежда съобщение за грешка и прекратява изпълнението на програмата.
- При успешна алокация функцията връща указател към `MemoryPool`, който е готов за използване от аллокатора.

#### 4.4.5.2. free\_memory\_pool()

Функцията `free_memory_pool(void* mem)` освобождава паметта, алокирана чрез `get_memory_pool_from_os()`.

- При успешно освобождаване се извежда съобщение за потвърждение.
- При възникване на грешка се извежда съобщение за неуспешно освобождаване на паметта.

#### 4.4.6. Организация на вътрешната памет

Тази секция описва как PMAD организира вътрешно паметта, как се добавят MemoryPool-и и как се изграждат SizeClass-овете и свободните списъци от блокове.

##### 4.4.6.1. attach\_new\_pool()

Функцията `attach_new_pool()` добавя нов MemoryPool към аллокатора PMAD. Тя получава блок памет от операционната система и поставя метаданните на MemoryPool в началото на този блок.

По този начин заглавката на MemoryPool се намира директно в самата управлявана памет, а останалата част от блока се използва за алокации.

| MemoryPool метаданни    | Памет за алокации              |
|-------------------------|--------------------------------|
| start, size, used, next | POOL_SIZE - sizeof(MemoryPool) |

*Фигура 4: Разделение в паметта на метаданните + наличната памет*

Новият MemoryPool се добавя в началото на списъка с пулове в PMAD, което позволява лесно разширяване на аллокатора без преместване на вече съществуващи блокове.

#### 4.4.6.2. split\_pool\_by\_percentage()

Функцията `split_pool_by_percentage()` разделя наличната памет в даден `MemoryPool` на `SizeClass`-и според конфигурацията, зададена от потребителя.

Както процентното разпределение, така и размерите на `SizeClass`-ите не са фиксирани. Те могат да бъдат динамично конфигурирани според конкретното приложение и неговите нужди.

Това позволява аллокаторът да бъде оптимизиран за различни натоварвания – например приложения, които използват предимно малки обекти, могат да заделят по-голям процент за малки блокове.

#### Гъвкавост на конфигурацията

- Размерите на `SizeClass`-ите се задават от потребителя.
- Процентите за разпределение също се задават от потребителя.
- Единственото ограничение е сумата на процентите да бъде точно 100%.

След като процентите бъдат валидирани, `MemoryPool` се разделя на области, като за всеки `SizeClass` се изчислява:

- Общата памет за класа (според процента).
- Броят блокове, които могат да се побрат.
- Реалният размер на блок (включително `BlockHeader`).

Метаданните на `SizeClass` (`block_size`, `free_list`, `total_blocks`, `allocated_blocks`) се съхраняват в структурата `PMAD`, а не вътре в `MemoryPool`. `MemoryPool` съдържа само суровата памет.

## Обща логическа организация

| SizeClass | User Size              | Процент                | Блокове (динамично) |
|-----------|------------------------|------------------------|---------------------|
| Class 0   | зададен от потребителя | зададен от потребителя | изчислява се        |
| Class 1   | зададен от потребителя | зададен от потребителя | изчислява се        |
| Class 2   | зададен от потребителя | зададен от потребителя | изчислява се        |
| Class N   | зададен от потребителя | зададен от потребителя | изчислява се        |

Фигура 5: Обща логическа организация

След разделянето на паметта се извиква `createBlock()` за всеки блок, за да се изгради свободният списък.

### `createBlock()`

Функцията `createBlock()` се извиква за създаването на отделен блок в рамките на даден `SizeClass`. Тя поставя метаданните `BlockHeader` в началото на всеки блок.

| BlockHeader      | Потребителска памет                       |
|------------------|-------------------------------------------|
| size_class, next | размер според конфигурацията на SizeClass |

Фигура 6: Структура на блок

Всеки нов блок се добавя в началото на свободния списък на съответния `SizeClass`:

- Новият блок сочи към предишната глава на `free_list`.
- `free_list` в PMAD се обновява да сочи към новия блок.

Това гарантира  $O(1)$  добавяне и изграждане на свободните списъци.

## 4.4.7. Алокация и освобождаване

### 4.4.7.1. Алокация

Алокацията на памет се извършва чрез функцията **PMAD\_alloc()**, която връща void\* указател към свободната памет за заявка с даден размер.

Функцията използва помощната функция **roundUP()** ([Таблица 7](#)) и таблицата **size\_class\_reference** ([Таблица 3](#)) за избор на подходящ SizeClass.

### 4.4.7.2. Функцията roundUp()

roundUP() приема размер на заявката (size) и го закръгля към най-малката стойност, която е кратна на ALIGNMENT.

Това гарантира правилно подравняване на всички блокове памет, което е изискване за ефективна работа на аллокатора и хардуера.

| Входен размер (bytes) | ALIGNMENT | Закръглен размер (bytes) |
|-----------------------|-----------|--------------------------|
| 1                     | 16        | 16                       |
| 17                    | 16        | 32                       |
| 33                    | 16        | 48                       |
| 50                    | 16        | 64                       |

*Фигура 7: Функция roundUP() - примерни входни и изходни стойности*

### 4.4.7.3. Пример на функциониране

Нека имаме следните конфигурации за SizeClass-овете и проценти за разделяне на пул памет:

```
size_t class_sizes[NUM_CLASSES] = {16, 32, 64, 128, 1024};
size_t percentages[NUM_CLASSES] = {10, 20, 20, 20, 30};
```

*Фигура 8: SizeClass конфигурация*

Таблицата `size_classes` (част от структурата `PMAD`) изглежда така:

```
(PMAD) {
    pool_head = 0x000000001049b8000
    size_classes = {
        [0] = { block_size = 16, free_list = 0x000000001049d1980, total_blocks = 3276,
                allocated_blocks = 0 }
        [1] = { block_size = 32, free_list = 0x00000000104a04c40, total_blocks = 4367,
                allocated_blocks = 1 }
        [2] = { block_size = 64, free_list = 0x00000000104a37f60, total_blocks = 2621,
                allocated_blocks = 0 }
        [3] = { block_size = 128, free_list = 0x00000000104a6b220, total_blocks = 1456,
                allocated_blocks = 0 }
        [4] = { block_size = 1024, free_list = 0x00000000104ab7980, total_blocks = 302,
                allocated_blocks = 0 }
    }
}
```

*Фигура 9: Част от структурата PMAD*

#### 4.4.7.4. Освобождаване на памет

Освобождаването се извършва чрез функцията **`pmad_free`**, която:

- Добавя блока обратно в началото на свободния списък на съответния `SizeClass`. Намира `SizeClass`-а от метаданните, стоящи в началото на блока ([Таблица 6](#)).
- Намалява броя на `allocated_blocks` за `SizeClass`-а с 1.
- Гарантира, че блоковете винаги са подравнени и свободният списък остава  $O(1)$  за операции.

#### 4.4.8. Обвивка и използване на алокатора



#### 4.4.8.1. Обвивка на алокатора (incPMAD)

Файловете **incPMAD.h** и **incPMAD.c** предоставят опростен интерфейс към PMAD, позволявайки на потребителя да алокира и освобождава памет чрез **pmad\_alloc(size)** и **pmad\_free(ptr)** без да се налага да подава структурата PMAD.

Функцията **pmad\_init(class\_sizes, percentages)** конфигурира глобален инстанс на алокатора с зададените размери и процентно разпределение, а **pmad\_destroy()** освобождава всички MemoryPool-и и нулира указателите.

Това прави PMAD лесен за използване и безопасен за управление на паметта, като скрива вътрешната сложност на SizeClass-овете и BlockHeader метаданните.

### 4.5. Реализация – обосновка за използвани технологични средства, алгоритми, литература, програмни приложения и др.

Реализацията на PMAD изисква комбинация от системно програмиране, задълбочено познаване на компютърните архитектури и прецизен подбор на алгоритми, които гарантират спазването на строги времеви ограничения и детерминистично поведение.

#### 4.5.1 Използвани технологични средства (Език и среда) Език за програмиране C (C99/C11)

Изборът на езика C е естествен за системен софтуер от този тип. Той предоставя:

- **Директен достъп до паметта** – чрез указатели (pointers) и адресна аритметика (pointer arithmetic), което е фундаментално за изграждането на BlockHeader метаданните и управлението на паметта.
- **Липса на Garbage Collector (GC)** – автоматичен GC би въвел непредвидими паузи (Stop-the-World), което противоречи на целите за детерминистично време на изпълнение.
- **Минимален overhead** – C програмите се компилират директно до машинен код без виртуална машина или междинен runtime слой.

## POSIX API (<sys/mman.h>, <time.h>)

За комуникация с операционната система се използва стандартът POSIX, който предоставя:

- `mmap()` и `mmapr()` – за директна работа с виртуалната памет на ниво страници.
- `clock_gettime(CLOCK_MONOTONIC)` – за измерване на време с наносекундна прецизност при бенчмаркинг.

## Компилатор и build система

- **GCC (GNU Compiler Collection)** – използван за компилация и оптимизация на кода.
- **Make** – автоматизация на процеса по компилация и свързване (linking).

## Развойна среда и дебъгване

- **Visual Studio Code (VSCode)** – използван като основна развойна среда.
- **LLDB (LLVM Debugger)** – използван за ниско-нивово дебъгване, инспекция на памет, регистри и стек рамки. LLDB позволява директен анализ на адреси и структури в реално време, което е особено важно при разработка на алокатор.

### 4.5.2. Използвани алгоритми и структури от данни

Алгоритмичният дизайн е подчинен на изискването за строго **O(1)** време за изпълнение.

## Slab Allocation (Алокация базирана на плочи)

Концепцията за Slab Allocation е въведена от Джеф Бонуик през 1994 г. в контекста на ядрото на SunOS [1]. Основната идея е паметта да се разделя на пулове от обекти с фиксиран размер (Size Classes), вместо да се управляват блокове с произволна големина.

Този подход елиминира външната фрагментация и позволява предвидимо време за алокация, което е особено важно при системи с изисквания за детерминистично поведение.

## LIFO Free List (Едносвързан списък като стек)

Използването на LIFO свободен списък е класически подход в управлението на динамична памет [2]. Свободните блокове се организират като стек, където новоосвободеният блок се поставя в началото на списъка.

Това гарантира  $O(1)$  добавяне и премахване, както и подобрява cache locality, тъй като наскоро използваните блокове е вероятно да се намират в кеша на процесора [7].

## $O(1)$ Lookup Table

Използването на предварително изчислена таблица за индексване е техника, често прилагана в реалновремеви системи за елиминиране на линейно търсене [2].

Таблицата `size_class_reference` позволява директно изчисляване на индекса на подходящия `SizeClass` чрез проста аритметична операция. Така се гарантира константна сложност  $O(1)$  без използване на цикли.

## Побитова аритметика за подравняване

Подравняването чрез побитови операции:

$(\text{size} + 15) \& \sim 15$

е стандартна техника в системното програмиране за избягване на скъпи операции като деление и модул [3].

Според оптимизационните насоки на Intel, побитовите операции се изпълняват за един тактов цикъл и са значително по-ефективни при управление на памет [7].

### 4.5.3 Програмни приложения и инструменти за анализ Бенчмарк система

Разработен е специализиран модул:

- Benchmark.c
- bench\_configs.c

Той измерва:

- Throughput
- Latency
- Брой алокации
- Memory overhead
- Поведение при различни конфигурации

### Визуализация (Web технологии)

Създаден е интерактивен модул:

allocator\_infographics.html

Използвани технологии:

- HTML5
- CSS3
- Chart.js

Това демонстрира съвместяване на low-level системно програмиране с high-level визуализация и анализ.

#### 4.5.4 Използвана литература и източници

За теоретичната основа и валидация на решенията са използвани следните източници:

1. Bonwick, J. (1994). *The Slab Allocator: An Object-Caching Kernel Memory Allocator*. Proceedings of the USENIX Summer Conference.
2. Kerrisk, M. (2010). *The Linux Programming Interface*. No Starch Press.
3. ISO/IEC 9899:2011 – Programming Languages — C (C11 Standard).
4. The Open Group Base Specifications Issue 7 (POSIX.1-2017).
5. Evans, J. (2011). *A Scalable Concurrent malloc(3) Implementation for FreeBSD (jemalloc documentation)*.
6. Google. *TCMalloc: Thread-Caching Malloc documentation*.
7. Intel® 64 and IA-32 Architectures Optimization Reference Manual. Intel Corporation.

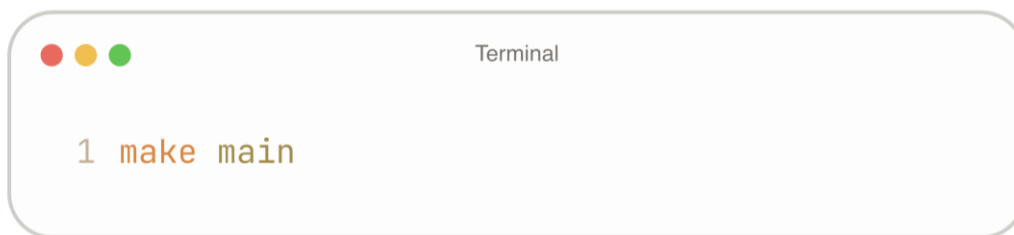
#### 4.6. Описание на приложението – инсталация, стартиране, използване и поддръжка

PMAD е реализиран като лека библиотека на езика C, предназначена за лесна интеграция в съществуващи проекти. Тя не изисква външни зависимости и не се нуждае от допълнителна инсталация извън стандартна компилация.

#### Инсталация и стартиране

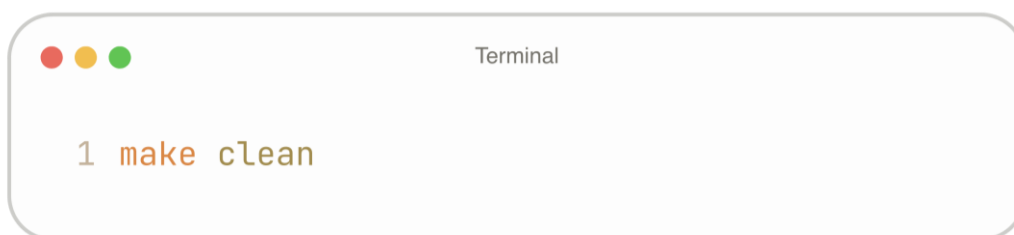
Компилацията на проекта се извършва чрез предоставения **Makefile** файл.

За генериране на основното демо приложение е необходимо в терминал да се изпълни командата:



*Фигура 10: Компилагционна команда*

За изчистване на генерираните обектни файлове и изпълними файлове се използва:



*Фигура 11: Команда за изтриване*

Проектът е съвместим с GCC/Clang компилатори в Linux среда.

## Интеграция и използване

За използване на PMAD в потребителски код е необходимо включване на публичния хедър файл:



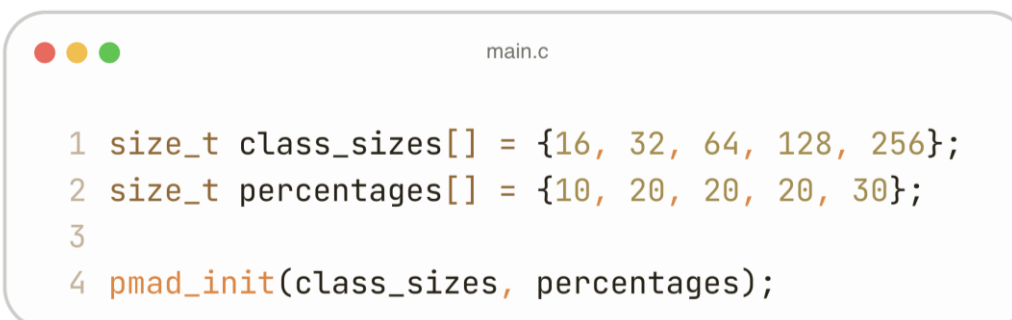
```
1 #include "incPMAD.h"
```

*Фигура 12: Главна библиотека*

Работата с алокатора се осъществява в три основни стъпки:

## 1. Инициализация

Дефинират се масиви с размерите на класовете и процентното разпределение (сумата трябва да бъде 100%):



```
1 size_t class_sizes[] = {16, 32, 64, 128, 256};
2 size_t percentages[] = {10, 20, 20, 20, 30};
3
4 pmad_init(class_sizes, percentages);
```

*Фигура 13: Инициализация на алокатора*

## 2. Заделяне и освобождаване на памет

Функциите `pmad_alloc()` и `pmad_free()` заместват стандартните `malloc()` и `free()`, като гарантират  $O(1)$  сложност.



```
1 void* my_data = pmad_alloc(40); // заделяне
2
3 if (my_data) {
4     pmad_free(my_data); // освобождаване
5 }
```

*Фигура 14: Алокация и освобождаване*

### 3. Унищожаване на алокатора

При приключване на програмата всички MemoryPool-и се освобождават:



```
1 pmad_destroy();
```

*Фигура 15: Унищожение*



## Пълен пример (алокация на масив от 6 елемента, принтиране и освобождаване)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include "incPMAD.h"
4
5 int main() {
6
7     /* 1. Конфигурация на класовете по размер */
8     size_t class_sizes[NUM_CLASSES] = {16, 32, 64, 128, 256};
9     size_t percentages[5] = {10, 20, 20, 20, 30};
10
11     /* Инициализация на алокатора */
12     pmad_init(class_sizes, percentages);
13
14     /* 2. Заделяне на памет за 6 цели числа */
15     int* something = pmad_alloc(sizeof(int) * 6);
16
17     /* 3. Запис на стойности в заделената памет */
18     int* ptr = something;
19     for (int i = 0; i < 6; i++) {
20         *ptr = i + 5;
21         ptr++;
22     }
23
24     /* 4. Извеждане на записаните стойности */
25     ptr = something;
26     for (int i = 0; i < 6; i++) {
27         printf(" %d", *ptr);
28         ptr++;
29     }
30     printf("\n");
31
32     /* 5. Освобождаване на паметта */
33     pmad_free(something);
34
35     /* 6. Унищожаване на алокатора */
36     pmad_destroy();
37
38     return 0;
39 }
```

## Поддръжка

Адаптирането на PMAD към различни типове приложения (игрови енджини, мрежови услуги, вградени системи) се извършва чрез:

- промяна на масивите `class_sizes` и `percentages`;
- настройка на константата `POOL_SIZE`.

Не се изисква модификация на вътрешната логика на библиотеката.

За анализ и проверка на производителността могат да се използват предоставените тестови файлове `benchmark.c` и `bench_configs.c` (може и да се види на Фигури: 1, 17-18)

## 4.7. Заключение

### 4.7.1. Основен резултат

Основният резултат от настоящия проект е успешната разработка на напълно функциониращ, детерминистичен алокатор на памет на езика C – **PMAD (Pool-based Memory Allocator with Determinism)**.

Системата изпълнява всички първоначално поставени цели:

- **Строга  $O(1)$  времева сложност** при заделяне и освобождаване на памет, без използване на цикли, линейно търсене или скъпи операции.
- **Нулеви системни извиквания по време на runtime**, благодарение на предварителна резервация на памет от операционната система.
- **Пълна предвидимост**, позволяваща предварително математическо изчисление на:
  - броя блокове във всеки `SizeClass`,
  - общата необходима памет,
  - процента `header overhead`.
- **Конфигурируема архитектура**, при която потребителят определя вътрешната структура чрез масивите `class_sizes` и `percentages`.

Тези характеристики правят PMAD подходящ за системи с изискване за ниска латентност и детерминирано поведение.

#### 4.7.2. Експериментални резултати

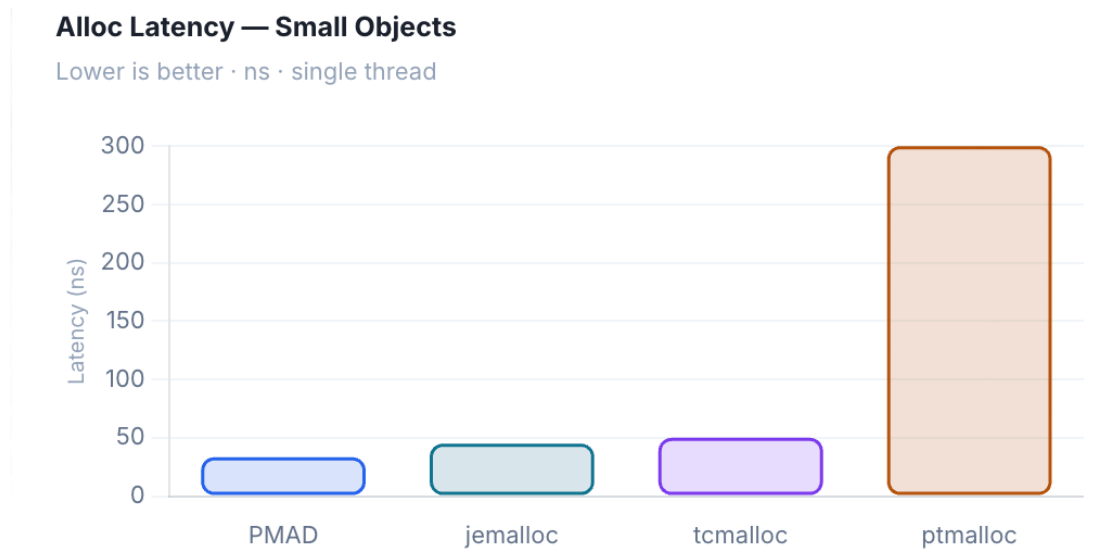
Производителността на PMAD беше анализирана чрез специализиран тестов модул (benchmark.c), като резултатите са обобщени в [Таблица 1](#) (**Сравнителен анализ на алокатори**) и визуализирани чрез диаграми.

Анализът показва следното:

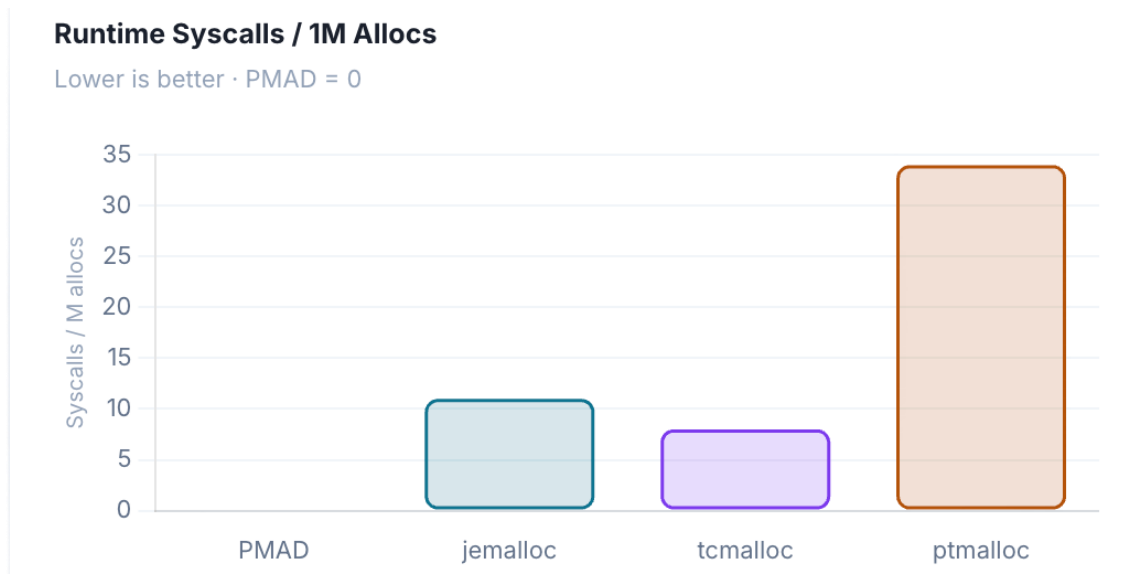
- При **малки обекти** PMAD демонстрира значително по-ниска латентност спрямо ptmalloc и конкурентни стойности спрямо jemalloc и tcmalloc.
- PMAD генерира **0 системни извиквания на 1М алокации**, което е съществено предимство при реалновремеви приложения.
- Латентностният jitter (p99–p50) е практически нулев, което потвърждава детерминистичния характер на архитектурата.
- При bulk throughput PMAD показва стабилна производителност при еднонишково натоварване.
- Header overhead е предварително изчислим и контролиран чрез конфигурацията.

### 4.7.3. Визуализация на резултатите

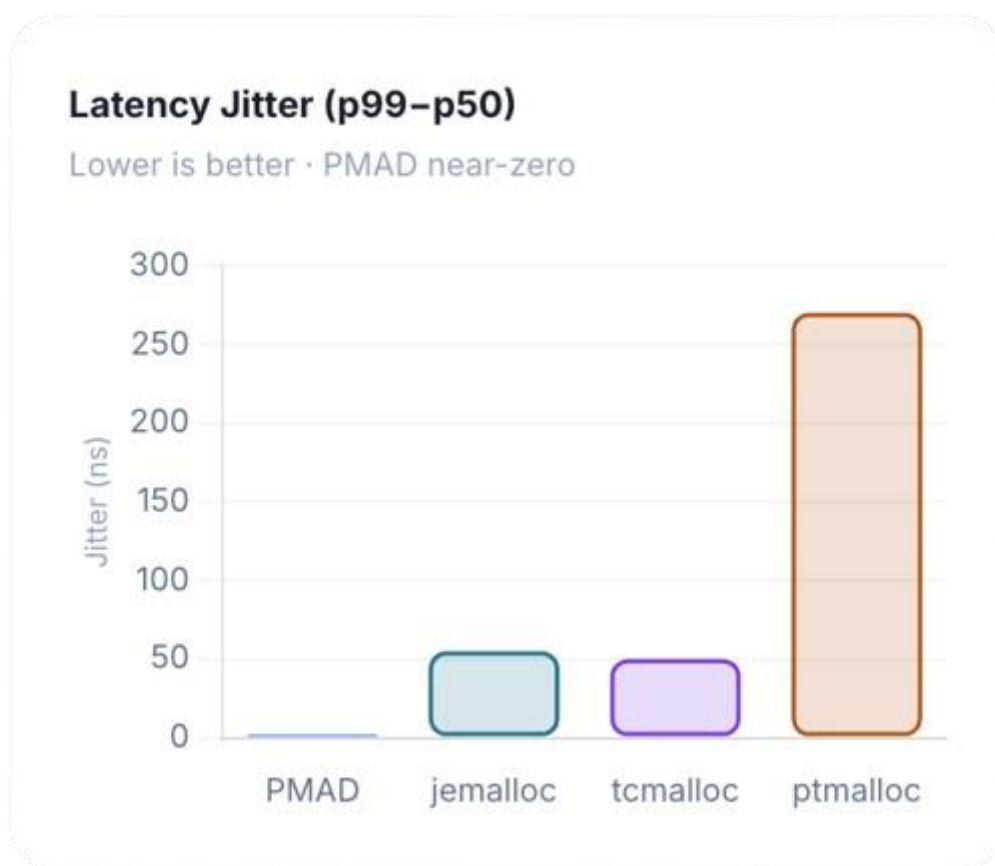
Фигура 17 – Латентност при малки обекти



Фигура 18 – Системни извиквания



## Фигура 19 – Latency Jitter (p99–p50)



Всички числови стойности са представени подробно в [Таблица 1](#), която служи като формална референтна база за сравнителния анализ.

### 4.7.4. Текущи приложения

Към момента PMAD функционира като:

- Концептуално демо на детерминистичен алокатор
- Бенчмарк платформа за сравнение със стандартния malloc
- Основа за интеграция в реални системи

Архитектурата е приложима в:

- Вградени системи и RTOS среди
- Системи за високочестотна търговия (HFT)

- Game engines (управление на често презаписвани малки обекти – частици, куршуми, ефекти)
- Нисколатентни мрежови услуги

#### 4.7.5. Възможности за бъдещо развитие

Въпреки стабилното ядро, архитектурата позволява значително надграждане:

##### 4.7.5.1. Поддръжка на многонишковост

Добавяне на:

- Thread-Local Storage (TLS) пулове
- Lock-free free list структури

Това би позволило безопасна работа в конкурентна среда без компромис в сложността.

##### 4.7.5.2. Динамично разширяване на пула

Имплементиране на контролирана логика за извикване на `attach_new_pool()`, описана в [раздел 4.4.6.1](#), при изчерпване на `SizeClass`.

Това би осигурило устойчивост при пиково натоварване, с компромис под формата на редки системни извиквания.

##### 4.7.5.3. Memory Safety механизми

- Проверка за double-free
- Валидация на указатели
- Canary стойности за защита от buffer overflow

##### 4.7.5.4. Разширено подравняване

Поддръжка на конфигурируемо подравняване (32B/64B) за SIMD/AVX оптимизации.

#### 4.7.6 Финална оценка

PMAD доказва, че чрез внимателен архитектурен дизайн и използване на pool-based модел може да бъде постигната:

- предвидима латентност,
- елиминиране на runtime системни извиквания,
- математически контрол върху ресурсите,
- стабилност при специфични натоварвания.

Сравнителният анализ (вж. [Фигура 1](#) и [Фигури 17-19](#)) PMAD предлага конкурентна и в някои повечето аспекти превъзхождаща производителност спрямо стандартните алокатори.